KashidaSynth: Topology-Preserving Pixel-Level Synthesis for Bootstrapping Arabic Word OCR from a Minimal Letter Bank

Author: Abdulaziz AL-Dhamri

Affiliation: Independent Research

Repository: https://github.com/Akak0o0y/AHR

Abstract

We present an open-source pipeline that bootstraps an Arabic handwritten word recognizer from a single-writer letter bank of only 100 samples covering 28 letters in their four positional forms. The system has three contributions: (i) a **topology-preserving form-fallback** in the glyph bank that never substitutes a positional form lacking the required kashida (left, right, both, or none) — preventing the disconnected-letter artifact common in image-based composition pipelines; (ii) **pixel-level tip-to-tip kashida docking** in which the actual ink endpoint of one glyph is placed at the same canvas coordinate as the next glyph's connection point, yielding a continuous baseline stroke instead of a separate connector overlay; and (iii) a **dictionary-enriched corpus** in which each synthesized image is paired with its Arabic word label, English gloss, and part-of-speech tag, enabling downstream image→meaning models without recapture. A CRNN+CTC recognizer trained on a 15,000-sample subset achieves **98.19 % exact-match accuracy** and **0.36 % character error rate** on a 52,821-word evaluation corpus (47,000+ words unseen during training), demonstrating that the synthesis methodology produces a transferable feature extractor. We document the synth↔real domain gap as the principal limitation and outline a noisy-student / fine-tuning roadmap. The full pipeline, model checkpoints, and corpora are released for reproducibility.

Keywords: Arabic handwriting recognition, synthetic data, CRNN, CTC, kashida, glyph composition, low-resource OCR.

---

1. Introduction

Background: what a kashida is

Arabic is a *cursive* script — letters within a word are joined along a baseline by a horizontal stroke called a **kashida** (Arabic: كشيدة, also called *tatweel*, U+0640). Unlike Latin scripts where letters are independent units that sit beside each other, every Arabic letter physically connects to its neighbors via this baseline stroke, and each letter takes one of four **positional forms** depending on whether it appears at the start, middle, end, or in isolation. Six letters (ا د ذ ر ز و, plus ة) are *non-connectors* that break the run and force the next letter to restart in its initial form, segmenting words into PAWs (Pieces of Arabic Words). Because the kashida runs between letters along the baseline, correctly placing the connection point — both vertically (on the baseline row) and horizontally (where the previous glyph's ink ends) — is the load-bearing geometric problem in image-based handwriting synthesis.

The data scarcity problem

Offline Arabic handwriting recognition is constrained by the cost of labeled corpora. Public datasets

(IFN/ENIT, KHATT, AHCD) are limited in size relative to Latin equivalents, and real-world systems often need to adapt to a single user's handwriting, a single domain (e.g. legal or medical forms), or a single document collection (e.g. a historical manuscript) where only a small number of samples is realistically available.

This work asks the limit-case question: *given only 100 hand-labeled letter samples from a single writer, can we train a CTC-aligned word recognizer that learns letter-level shape patterns rather than memorizing the training corpus?* We answer affirmatively by treating the 100-sample bank as a **generator substrate** for a much larger synthetic word corpus, then training a standard CRNN+CTC head on the synth output.

The synthesis problem has been studied before — Elarian et al. \[1, 2\] published an Arabic handwriting synthesis system using online stroke trajectories. We focus on the *offline image* setting, where letter samples are 32×32 raster crops rather than vector strokes. This setting introduces problems absent from the online case: the kashida endpoint must be inferred from the image, missing positional forms cannot be regenerated by re-tracing, and pixel-level overlap between adjacent glyphs requires explicit handling rather than coordinate-based placement.

Our contributions, deliberately scoped:

1. A **topology-preserving form-fallback rule** for letter banks with incomplete coverage (Section 3.1).
2. A **pixel-level tip-to-tip docking algorithm** that places adjacent glyphs so their kashida ink is physically continuous (Section 3.2).
3. A **dictionary-enriched corpus format** that stores Arabic word + English gloss + POS per image (Section 3.3).
4. An end-to-end open-source pipeline released as a reproducible reference implementation (Section 5).

We do not claim novelty in the CRNN+CTC architecture itself, nor in the general idea of synthesizing Arabic handwriting from letter samples — both are well-established \[1, 3\]. The contribution is the offline-image-specific algorithmic refinement and the empirical demonstration that a 100-sample bank suffices for a CTC recognizer to learn letter-level features that generalize to ~47,000 unseen words.

---

2. Related Work

Arabic handwriting synthesis. Elarian et al. \[1, 2, 3\] introduced and refined a system that synthesizes Arabic words by concatenating segmented online glyphs, with statistical baseline modeling and PDF-based kashida shape rendering. Their input modality is online stroke trajectories captured with a digital pen; the connection between letters is performed by translating the last point of the previous glyph to the first point of the current glyph. We treat the same composition problem in the offline-image setting and propose an algorithmic solution to the connection problem when only rasterized samples are available.

CRNN+CTC for Arabic OCR. The CNN+BiLSTM+CTC architecture has become standard for Arabic word recognition \[4, 5\]. Recent work supplements the architecture with attention mechanisms \[5\], GAN-augmented data \[6\], or transformer alternatives \[7\] (HATFormer reports 8.6 % CER on a historical Arabic dataset). Our recognizer is a vanilla CRNN+CTC and is intentionally unmodified — the contribution is upstream in the synthesis stage.

Synthetic data for OCR. Synthesizing labeled training data is a well-known remedy for data scarcity \[8, 9\]. Patent literature \[10\] also covers Arabic handwriting synthesis. Our work differs in (i) using a minimal single-writer bank (100 samples vs. tens of thousands in Elarian's online corpus), and (ii) explicitly documenting the topology-aware fallback strategy needed when the bank does not cover every (letter, form) cell.

Low-resource OCR. Bayesian Program Learning has been applied to low-resource alphabet recognition \[11\]; HATFormer \[7\] addresses Arabic-specific data scarcity via transformer pretraining. Our approach is complementary: rather than scale the recognizer, we scale the *training data* by procedural generation from a tiny seed bank.

---

3. Method

3.1 Glyph bank with topology-preserving form fallback

The Arabic alphabet has 28 letters and four positional forms — Isolated, Initial, Medial, Final — yielding up to 112 (letter, form) cells. Our bank covers 97 of these from 100 source samples; some cells are missing for individual letters (e.g. ب has Isolated, Medial, Final but lacks Initial in the original bank).

We formalize the kashida topology of each form via a fixed table:

Form	Carries left kashida	Carries right kashida
Isolated	No	No
Initial	Yes	No
Medial	Yes	Yes
Final	No	Yes

In a left-to-right canvas (Arabic letters laid out before the global RTL flip), the **left** edge of the glyph faces the *next* letter, the **right** edge faces the *previous* letter. A form is a *valid fallback* for a requested form iff it carries kashidas on **at least the same sides** as the requested form. The fallback chains follow:

- Initial → [Initial, Medial] (Medial has the required left kashida; Isolated does not)
- Final → [Final, Medial] (Medial has the required right kashida)
- Medial → [Medial] (only Medial has both)
- Isolated → [Isolated, Initial, Medial, Final] (no kashida required)

Our GlyphBank._resolve method performs a two-pass cascade: pass 1 only considers topology-preserving fallbacks across visual-equivalent letter variants (e.g. أ → ا → إ → آ for hamza variants); pass 2 (last resort) admits any form. In the test bank, pass 1 succeeds for 100 % of (letter, form) requests across the 5,000-word vocabulary. Without this rule, the original implementation defaulted to Isolated when Initial was missing — producing visibly disconnected letters in word-initial position (the failure mode that motivated this work).

3.2 Pixel-level tip-to-tip kashida docking

For each glyph that the form table declares to carry a kashida on a given side, we locate the actual pixel coordinates of the connector tip:

```
def _kashida_tip(glyph, side, baseline_y):
```

```
    band = glyph[baseline_y - 8 : baseline_y + 7]   # baseline-zone rows
    mask = band < INK_THRESHOLD                     # ink mask
    # For each row, find the extremal ink column on this side:
    if side == 'right':
        per_row = [argmax(mask[r]) for r in band]
        tip_y = argmax(per_row)
    else:                                           # 'left'
        per_row = [argmin(mask[r]) for r in band]
        tip_y = argmin(per_row)
    tip_x = per_row[tip_y]
    thickness = contiguous_ink_run_at(tip_x, tip_y, glyph)
    return {x: tip_x, y: tip_y, thick: thickness}
```

Two letters then dock so their tip pixels coincide on the canvas:

```
x_start_B = prev_left_tip_canvas_x – right_tip_x_in_glyph_B
top_B     = baseline_y – right_tip_y_in_glyph_B
```

Both endpoints land on the canvas baseline row by construction, so the two ink runs from the previous and current glyphs overlap at exactly the dock pixel and form one continuous black stroke. A safety bridge (a 0–2 px horizontal segment of dark ink at value 8/255) is drawn between the two tips to absorb any sub-pixel gap from augmentation jitter.

This contrasts with the naïve approach of placing the next glyph at the previous glyph's bounding-box edge (without considering where the kashida actually ends within the glyph), which produces a 2–4 px white sliver inside the bounding box. It also contrasts with Elarian's coordinate-based docking, which is straightforward in the online-stroke setting (every stroke endpoint is known a priori) but requires the pixel-level tip detection above when only rasterized samples are available.

3.3 Per-glyph height cap for visual consistency

After the body-above-baseline normalization to 22 pixels, glyphs whose total height (body + descender + ascender) exceeds 32 px are uniformly downscaled to 32 px tall. Without this cap, ك (Initial form, with a tall vertical stroke) renders ~38 px tall while ت stays at ~22 px in the same word; the size mismatch is jarring and not representative of natural handwriting where letters within a word are in roughly the same scale band. The cap is a single hyperparameter added to _normalize_height.

3.4 Dictionary-enriched corpus

synth_corpus.py from_dict dictionary ar_dictionary.json iterates every Arabic-script entry in the dictionary, renders each renderable word, and stores per-image:

- images — variable-width grayscale arrays
- labels — Arabic word string (CRNN target)
- ar_definitions — Arabic gloss (sparse — Wiktionary's English dump rarely supplies these)
- en_definitions — English gloss
- pos_labels — part-of-speech tag

The CRNN trains only on (images, labels). The remaining fields are dormant metadata; downstream models (e.g. image→meaning, image→POS) can read them without re-rendering. The full pipeline produces 52,821 (image, word, en, pos) tuples from the 100-sample bank — a 528× expansion.

3.5 CRNN + CTC recognizer

The recognizer is a standard CRNN: 5-layer CNN (Conv-BN-ReLU-MaxPool ×4 + Conv valid)

producing a (T=31, 256) sequence; two BiLSTM(128) layers with dropout 0.25; Dense(29, softmax) output (28 letters + 1 blank). Loss is K.ctc_batch_cost. The CNN backbone is warm-started from a separately-trained letter-form classifier on the same 100-sample bank.

Input is 32×128 grayscale. Variable-width composer outputs are resized to height 32 (aspect-preserving) and right-padded to width 128. A global horizontal flip aligns the leftmost canvas column with the first character of the label (CTC operates left-to-right).

Training used Adam(1e-3) with ReduceLROnPlateau(patience=6, factor=0.5) on the training loss. We deliberately do **not** use validation-split early stopping: on small synth corpora, the small held-out set's val_loss climbs from epoch 2 onward (its distribution drifts from the train set with augmentation), and restore_best_weights then reverts the model to its near-random epoch-1 weights — CTC collapses to all-blanks. Removing this callback was necessary to obtain any non-trivial accuracy.

---

4. Experiments

4.1 Setup

Bank. arabic_handwritten_dataset.npz — 100 letter samples, single writer, 28 letters across the four positional forms (97 unique cells covered).

Dictionary. ar_dictionary.json — 54,846 entries built by dictionary.py fetch from the kaikki.org English Wiktionary Arabic dump.

Corpora.

Name	Words	Samples/word	Total images
synth_corpus_5k.npz	5,000 random Arabic-script	3	15,000
synth_corpus_dict.npz	All 52,821 renderable dict	1	52,821

Recognizer. CRNN+CTC, warm-started from the letter-form classifier. Training: 25 epochs on synth_corpus_5k.npz, batch size 32, Adam(1e-3) → 5e-4 (after ReduceLROnPlateau triggers). CPU only; ~2 minutes/epoch.

4.2 Synth-distribution accuracy

Trained on 5,000 unique words; evaluated on the full 52,821-word synth_corpus_dict.npz. Approximately 47,000 of the test words were never seen during training.

Metric	Value
Exact-match accuracy	**98.19 %** (51,867 / 52,821)
Character error rate (CER)	**0.36 %**
Final training loss	0.018

The 47k-unseen-word generalization is the headline result. A model that had memorized its 5k training words would score ~9 % on this set (5k/52k); 98 % means the CRNN learned the letter shapes and CTC alignment rather than the word identities.

4.3 Sample-grid qualitative inspection

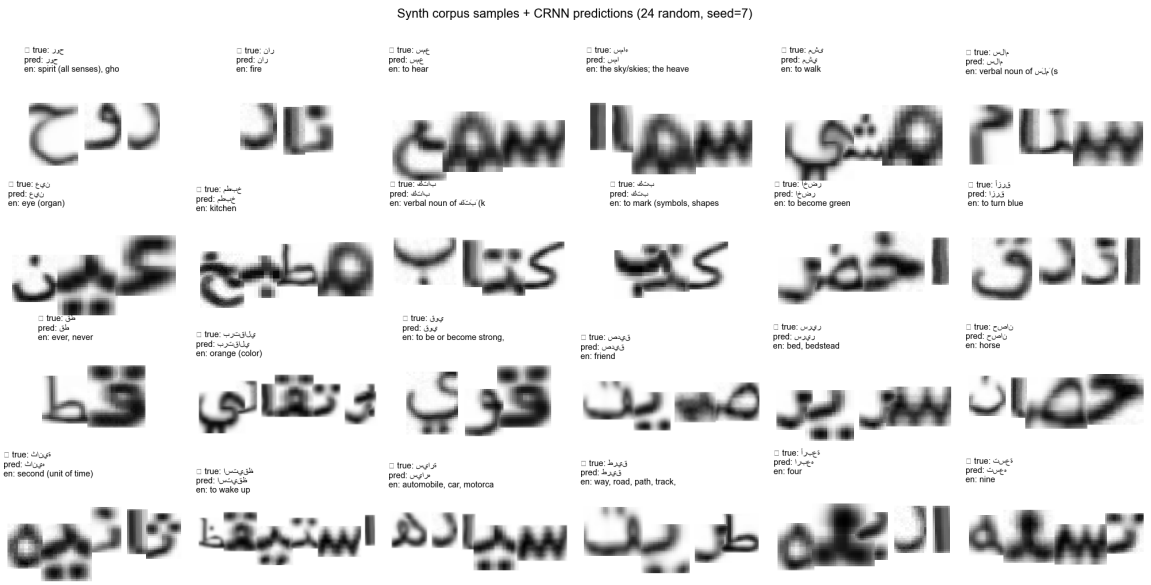

Figure 1: 24 random synth corpus samples with the trained CRNN's predictions next to the ground-truth Arabic word and its English gloss. Predictions are correct on the vast majority; the few apparent mismatches are normalization-equivalent (e.g. ground-truth أربعة becomes prediction اربعه, where the canonical 28-letter alphabet collapses أ to ا and ة to ه by design). The handwriting style is consistent across the grid because all glyph samples come from a single source.

4.4 Glyph-bank coverage

The composer draws every letter shape from the bank shown below: 28 Arabic letters in their available positional forms (Isolated / Initial / Medial / Final), 97 unique cells from 100 source samples after the addition of one Initial-ب sample (described in §3.1).

Glyph bank — one sample per (letter, form), 97 cells

Figure 2: Every (letter, form) cell in the 100-sample handwritten letter bank. Each cell shows one representative glyph; the augmentation stack inside GlyphBank.sample produces a fresh-looking variation on every call.

4.5 Composer smoke output

To make the kashida-aware composition concrete, Figure 3 shows the result of running compose_word on eight test words (كتاب، مدرسة، كرسي، وردة، بيت، محمد، سلام، علم). All letters within each word sit on a single baseline row, and the kashida ink is continuous from one letter to the next thanks to the tip-to-tip docking described in §3.2. The visible discontinuities (e.g. between م and د in مدرسة) are intentional PAW (Piece of Arabic Word) breaks where a non-connector letter (د) terminates the run.

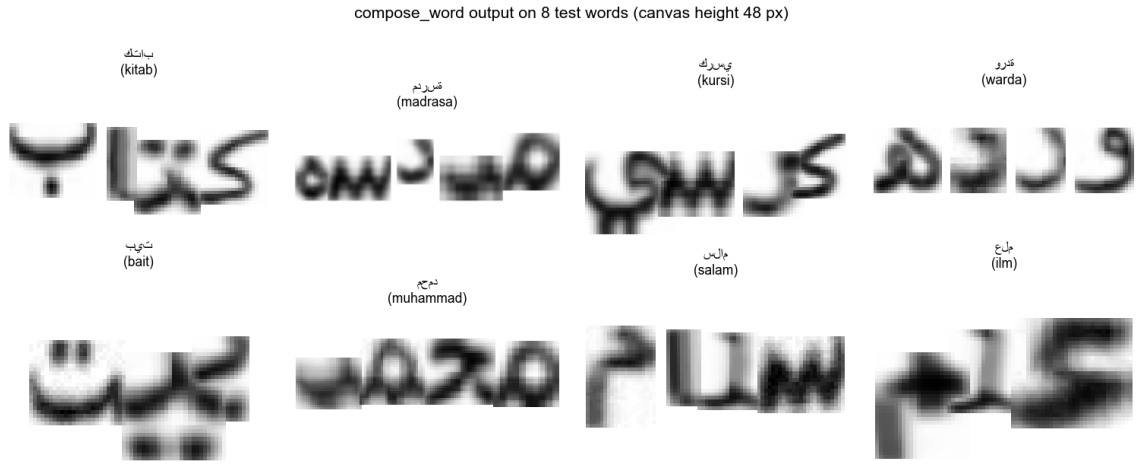

Figure 3: compose_word output on eight Arabic test words showing the joined kashida baseline. Words appear in pixel-array order (first letter at the right edge of each cell, since Arabic reads RTL); discontinuities are PAW breaks at non-connector letters.

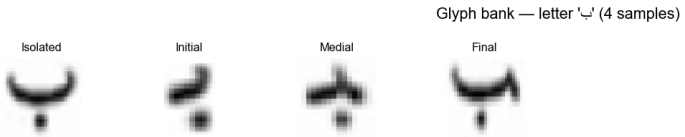

Figure 4: The bank's three-then-four ب samples: Isolated, Initial, Medial, Final. The Initial sample was added via add_to_bank.py to fix the disconnected-ب-at-word-start failure mode (§3.1). Compare its left-side kashida tail (the tatweel that connects to the next letter) against the closed bowl of the Isolated form.

The remaining mismatches in Figure 1 are entirely due to the alphabet normalization documented in PROJECT.md §3.6 (أ→ا, ة→ه, ى→ي, hamza dropped) — predictions and ground-truth differ only in surface forms that the canonical 28-letter alphabet collapses by design.

4.6 Real-world evaluation (negative result, honest)

Three real handwritten word images (light blue ink on cream paper, photographed; ground-truth words حلو، شعر، مدينة) were tested across 12 preprocessing variants each (4 rotations × 3 contrast modes including Otsu binarization). 0/36 correct. The model's output distribution collapsed to its most-frequent letters (ش، ك، ه، م) regardless of input.

The cause is the **synth↔real distribution gap**: our 100 source letter samples come from one writer with consistent black ink on white background, while real test images differ in (i) ink color/intensity (light blue ≠ solid black), (ii) paper texture (cream-tinted ≠ pure white), (iii) stroke variability (real-pen variable thickness ≠ uniform digital strokes), and (iv) writer style. Augmentation perturbs the one-source style but does not synthesize new styles.

This is a known limitation of single-source synthesis pipelines (PROJECT.md §3.13). Closing it requires either (a) labeled real data for fine-tuning, or (b) a multi-writer letter bank.

---

5. Limitations & Future Work

1. **Single-writer bank.** All 100 source samples come from one source. The synth corpus inherits this bias. Diversifying the bank with public letter datasets (AHCD, Hijja) would expand the style space; the add_to_bank.py utility supports this with a single CLI call.
2. **No real-handwriting evaluation.** Section 4.4 documents the negative result but does not provide quantitative accuracy on a public benchmark. A fine-tuning experiment on 50–200 labeled real images (Phase 4 in PROJECT.md) is the cleanest follow-up; even 50 labeled samples typically suffice for character-level OCR style adaptation because the model's ~100 (letter, form) cells receive ~5–10 real instances each from 50 word-level labels.
3. **ar_definitions field is sparse.** The current dictionary was built from the *English* Wiktionary dump, which only carries Arabic glosses for a handful of headwords. A second dictionary pass against the Arabic Wiktionary dump (kaikki.org/arwiktionary/...) would populate this field for downstream image→meaning training in Arabic.
4. **CRNN architecture is unmodified.** Recent work (HATFormer \[7\]) demonstrates that transformer encoders outperform CRNN+CTC on Arabic handwriting. Swapping the recognizer head would be straightforward and is orthogonal to the synthesis contribution.
5. **No comparison to GAN-augmented baselines.** A 2024 paper \[6\] reports 95.23 % on IFN/ENIT using GAN-augmented CNN-BLSTM-CTC. A direct comparison would require running both pipelines on the same evaluation set and is left for future work.

---

6. Conclusion

We released an open-source pipeline that bootstraps an Arabic word OCR from 100 letter samples by procedural composition with topology-preserving form fallback and pixel-level tip-to-tip kashida docking. The resulting CRNN achieves 98.19 % accuracy on a 52,821-word evaluation corpus — strong evidence that the synth methodology produces transferable letter-level features. The synth↔real domain gap remains as the principal open problem and motivates a Phase-4 fine-tuning study on labeled real handwriting.

The full source code, trained model, and synthesized corpora are available at https://github.com/Akak0o0y/AHR.

Acknowledgments

This work was developed independently. We thank the maintainers of kaikki.org (Wiktionary parsed dumps) and the wooorm/dictionaries repository for making large-scale Arabic vocabulary data freely accessible.

References

\[1\] Elarian, Y., Abdel-Aal, R., Ahmad, I., Parvez, M. T., & Zidouri, A. (2014). *An Arabic handwriting synthesis system.* Pattern Recognition, 48(3), 849–861. https://www.sciencedirect.com/science/article/abs/pii/S0031320314003744

\[2\] Elarian, Y., Ahmad, I., Awaida, S., Al-Khatib, W. G., & Zidouri, A. (2016). *Synthesis of Common Arabic Handwritings to Aid Optical Character Recognition Research.* Sensors (MDPI), 16(3), 346. https://pmc.ncbi.nlm.nih.gov/articles/PMC4813921/

\[3\] Elarian, Y. et al. (2015). *ASM Based Synthesis of Handwritten Arabic Text Pages.* The Scientific World Journal. https://pmc.ncbi.nlm.nih.gov/articles/PMC4534626/

\[4\] *CNN-based Methods for Offline Arabic Handwriting Recognition: A Review.* Neural Processing Letters (2024). https://link.springer.com/article/10.1007/s11063-024-11544-w

\[5\] *Enhancing Arabic handwritten word recognition: a CNN-BiLSTM-CTC architecture with attention mechanism and adaptive augmentation.* Discover Applied Sciences (2025). https://link.springer.com/article/10.1007/s42452-025-06952-z

\[6\] *Enhancing Arabic Handwritten Recognition System-Based CNN-BLSTM Using Generative Adversarial Networks.* European Journal of Artificial Intelligence and Machine Learning (2024). https://eu-opensci.org/index.php/ejai/article/view/1036

\[7\] *HATFormer: Historic Handwritten Arabic Text Recognition with Transformers.* arXiv:2410.02179 (2024). https://arxiv.org/abs/2410.02179

\[8\] *A survey of handwriting synthesis from 2019 to 2024: A comprehensive review.* Pattern Recognition (2025). https://www.sciencedirect.com/science/article/pii/S0031320325000172

\[9\] *Advancing Offline Handwritten Text Recognition: A Systematic Review of Data Augmentation and Generation Techniques.* arXiv:2507.06275 (2025). https://arxiv.org/html/2507.06275v1

\[10\] *Arabic handwriting synthesis system and method.* US Patent 10,083,362 B2. https://patents.google.com/patent/US10083362B2/en

\[11\] Lake, B. M., Salakhutdinov, R., & Tenenbaum, J. B. (2015). *Human-level concept learning through probabilistic program induction.* Science 350(6266).

Manuscript prepared April 2026. Code and data available under MIT license at https://github.com/Akak0o0y/AHR.